

Extract values from arrays and properties from objects into clean, readable variables — one of the most used features in modern JavaScript.

What is Destructuring?

Destructuring is a syntax that lets you **unpack values from arrays or properties from objects** into individual variables — in a single, clean line.

The Old Way vs Destructuring

```
// Old way - verbose and repetitive
const colors = ['red', 'green', 'blue', 'orange'];
const first = colors[0];
const second = colors[1];
const third = colors[2];
console.log(first, second, third); // red green blue

// Destructuring - clean and expressive
const [first, second, third] = colors;
console.log(first, second, third); // red green blue
```

Part 1 — Array Destructuring

Basic Syntax

```
const [a, b, c] = [1, 2, 3];
// a = 1, b = 2, c = 3
```

The **position** in the pattern maps to the **index** in the array.

Destructuring Strings

Strings are iterable, so destructuring works on them too:

```
const str = "hello";  
const [a, b, c] = str;  
console.log(a, b, c); // h e l
```

Skipping Elements

Use empty commas to skip positions you don't need:

```
const nums = [1, 2, 3, 4, 5];  
const [first, , third, , fifth] = nums;  
console.log(first, third, fifth); // 1 3 5
```

Each comma represents one skipped element.

Rest in Destructuring

Combine skipping with rest to collect everything after certain elements:

```
const nums = [1, 2, 3, 4, 5];  
const [, , ...rest] = nums;  
console.log(rest); // [3, 4, 5] - skip first two, collect the rest
```

Collect all elements into a named array:

```
const [...all] = [1, 2, 3, 4, 5];  
console.log(all); // [1, 2, 3, 4, 5]
```

Rule: Rest must always be the last element in a destructuring pattern.

Default Values

If the array has fewer elements than the pattern, the extra variables would be `undefined`. Default values prevent this:

```
const [a, b, c = 0] = [1, 2];  
console.log(a, b, c); // 1 2 0 ← c used its default
```

Default values only apply when the extracted value is `undefined` — NOT when it's `null`:

```
const [x = 10, y = 20] = [undefined, null];  
console.log(x, y); // 10 null  
// x used default (value was undefined)  
// y did NOT use default (null is a real value in JS)
```

Critical distinction: `undefined` triggers default values. `null` does not.

Swapping Variables

Destructuring makes variable swapping elegant — no temp variable needed:

```
let x = 10, y = 20, z = 30;  
console.log(x, y, z); // 10 20 30  
  
[x, y, z] = [z, x, y];  
console.log(x, y, z); // 30 10 20
```

Old approach required a temporary variable: `let temp = x; x = y; y = temp;` — destructuring eliminates this.

Nested Array Destructuring

Destructure arrays inside arrays by mirroring the structure:

```
const nested = [1, [2, 3], 4];  
  
// Simple – extract the inner array as a whole  
const [a, b, c] = nested;  
console.log(a, b, c); // 1 [2,3] 4
```

```
// Deep – extract specific elements from the inner array
const [a, [, b], c] = nested; // skip 2, get 3
console.log(a, b, c); // 1 3 4
```

The inner `[, b]` pattern matches the inner array `[2, 3]`, skipping `2` and assigning `3` to `b`.

Deeply Nested Arrays

Mirror the exact nesting structure of your data:

```
const deep = [1, [2, [3, [4, 5]]]];

// Extract x = 4, y = 5
const [, [, [, [x, y]]]] = deep;
console.log(x, y); // 4 5
```

Mixed Nested Destructuring with Rest

```
// first = 1, second = 2, innerRest = [3,4], last = 5
const [first, [second, ...innerRest], last] = [1, [2, 3, 4], 5];
console.log(first, second, innerRest, last); // 1 2 [3,4] 5
```

Destructuring Function Return Values

Functions often return arrays. Destructuring makes consuming them clean:

```
function mixedValues(x) {
  return [1, undefined, ['a', 'b'], function() {
    console.log("hello"); }, x];
}
```

```
// x=10(default), y='a', z=function, data=30
const [, x = 10, [y], z, data] = mixedValues(30);

console.log(x);    // 10 (was undefined, used default)
console.log(y);    // 'a' (first element of inner array)
console.log(data); // 30 (passed argument)
z();               // "hello" (calling the extracted function)
```

Part 2 — Object Destructuring

Basic Syntax

```
const user = {
  fname: "omar",
  age: 30,
  email: "omar@gmail.com"
};

const { fname, email } = user;
console.log(fname, email); // omar omar@gmail.com
```

Unlike arrays (where position matters), object destructuring matches by **property name**.

Renaming (Aliasing)

Sometimes you want to use a different variable name than the property name:

```
const { fname: userName, email: userEmail } = user;

console.log(userName, userEmail); // omar omar@gmail.com
console.log(fname);               // ReferenceError - 'fname' doesn't
exist as a variable
```

Syntax: `{ originalKey: newVariableName }`

Default Values in Objects

```
const { a = 10, b = 20 } = { a: undefined, b: null };

console.log(a, b); // 10 null
// a used default (undefined)
// b kept null (null is a real value, not undefined)
```

You can combine renaming AND defaults:

```
const { fname: userName = 'unknown' } = user;
console.log(userName); // "omar" (or "unknown" if fname was undefined)
```

Default Value for Missing Object

If a property might not exist at all on the object, set a default for it:

```
const { fname, address = {} } = user;
console.log(address.city); // undefined (safe - no error thrown)
```

Without `address = {}`, if `user` had no `address` property, the next line trying to access `address.city` would throw a `TypeError`.

Nested Object Destructuring

Mirror the object structure in the pattern:

```
const user = {
  fname: "omar",
  age: 30,
  address: {
    city: "cairo",
```

```

    country: "Egypt"
  }
};

const { fname, address: { city, country } } = user;

console.log(fname); // omar
console.log(city); // cairo
console.log(country); // Egypt
console.log(address); // ReferenceError - 'address' is NOT extracted
as a variable here

```

When you write `address: { city, country }`, you're using `address` as a path to navigate to, not assigning it as a variable. To get both the nested values AND the `address` object itself, destructure it twice:

```

const { fname: userName = 'unknown', address, address: { country } }
= user;

console.log(userName); // omar
console.log(address); // { city: "cairo", country: "Egypt" }
console.log(country); // Egypt

```

Destructuring in for...of Loops

Extremely useful when iterating over arrays of objects:

```

const users = [
  { id: 1, name: "omar", email: "omar@gmail.com" },
  { id: 2, name: "ali", email: "ali@gmail.com" },
  { id: 3, name: "mona", email: "mona@gmail.com" },
  { id: 4, name: "ahmed", email: "ahmed@gmail.com" },
];

// Without destructuring
for (const user of users) {
  console.log(user.name);
}

```

```
// With destructuring - clean and direct
for (const { name } of users) {
  console.log(name); // omar, ali, mona, ahmed
}

// Destructure multiple properties
for (const { name, email } of users) {
  console.log(name, email);
}
```

Reassigning Existing Variables

You can destructure into variables that already exist. The trick: wrap the whole expression in parentheses — otherwise JavaScript thinks `{` is a block statement.

```
let name = "original";

// Parentheses required when no declaration keyword (let/const/var)
({ name } = { name: "updated" });

console.log(name); // "updated"
```

Without the parentheses: `{ name } = { name: "updated" }` — JavaScript parses `{name}` as a code block and throws a `SyntaxError`.

Destructuring in Function Parameters

One of the most common real-world uses — destructuring directly in function signatures:

```
// Without destructuring
function greet(user) {
  console.log(user.name, user.age);
}
```



```
// With destructuring – self-documenting, clean
function greet({ name, age }) {
  console.log(name, age);
}

greet({ name: "omar", age: 30 }); // omar 30
```

With defaults:

```
function createUser({ name = "Guest", role = "user", age = 18 } = {})
{
  return { name, role, age };
}

createUser({ name: "omar" }); // { name: "omar", role: "user", age:
18 }
createUser();                 // { name: "Guest", role: "user", age:
18 }
```

The `= {}` at the end means the entire parameter has a default of an empty object — so calling the function with no arguments doesn't throw an error.

Array vs Object Destructuring — Key Differences

	Array	Object
Match by	Position (index)	Property name
Rename	Not applicable	<code>{ key: newName }</code>
Skip elements	Yes (empty commas <code>,</code>)	Yes (just omit the key)
Rest	Yes (<code>...rest</code>)	Yes (<code>...rest</code>)
Default values	Yes	Yes
<code>null</code> triggers default	No	No
<code>undefined</code> triggers default	Yes	Yes

Complete Summary

```
Array:    const [a, b, c]           = array
Object:   const { x, y, z }         = object
Rename:   const { key: newName }    = object
Default:  const [a = 10]            = array
Nested:   const [a, [b, c]]         = nestedArray
Nested:   const { a: { b } }        = nestedObject
Skip:     const [, , third]         = array
Rest:     const [first, ...rest]     = array
Swap:     [x, y] = [y, x]
Loop:     for (const { name } of users)
```

Abdullah Ali

Contact : +201012613453